

INTRODUCTION TO OPERATING SYSTEM

1 What is an Operating System?

Operating System ek **system software** hai jo computer ke hardware aur user ke beech interface ka kaam karta hai.

Jab computer start hota hai, sabse pehle OS memory me load hota hai. Uske baad hi hum kisi application ko run kar sakte hain.

Without OS:

- Hardware directly use nahi ho sakta
- Programs execute nahi ho sakte

Isliye OS ko “Resource Manager” aur “Control Program” bhi kaha jata hai.

2 Goals of the Operating System

Operating System ke main goals hote hain:

1. Convenience

User ke liye computer use karna easy banana.

2. Efficiency

Hardware resources ka maximum use karna.

3. Ability to Evolve

Time ke saath new features add kar sake.

3 Types of Operating Systems

Operating System alag-alag types ke hote hain:

1. Batch Operating System

Programs batches me execute hote hain.
User interaction nahi hota.

2. Multiprogramming Operating System

Ek time me multiple programs memory me load hote hain.

3. Time-Sharing Operating System

Multiple users ek system share karte hain.
Fast response milta hai.

4. Real-Time Operating System (RTOS)

Fixed time ke andar response dena zaroori hota hai.

5. Distributed Operating System

Multiple computers ek network me connected hote hain aur ek system ki tarah kaam karte hain.

4 Main Functions of an Operating System

Operating System ke major functions:

♦ 1. Process Management

- Process create karna
 - Process schedule karna
 - Process terminate karna
 - Context switching manage karna
-

♦ 2. Memory Management

- Memory allocate karna
 - Memory free karna
 - Virtual memory manage karna
-

♦ 3. File System Management

- Files create/delete
- Directory structure maintain karna

- File permissions control karna
-

◆ 4. Device Management

- I/O devices control karna
 - Device drivers use karna
 - Buffering aur caching handle karna
-

◆ 5. Security and Protection

- User authentication
 - Access control
 - Data protection
-

◆ 6. Networking

- System ko network se connect karna
 - Data transfer manage karna
-

5 Operating System Services

OS users aur programs ko services provide karta hai:

1. Program Execution
 2. I/O Operations
 3. File System Manipulation
 4. Communication (IPC)
 5. Error Detection
 6. Resource Allocation
 7. Accounting
 8. Protection and Security
-

6 System Software vs Application Software

System Software

- Hardware ko control karta hai
- Background me run hota hai
- Example: OS, Compiler

Application Software

- User tasks perform karta hai
- User ke liye visible hota hai
- Example: Browser, MS Word

Main Difference:

System software system ko chalata hai,

Application software user ka kaam karta hai.



Important Theory Lines for Exam

- An operating system is a resource allocator and control program.
 - OS acts as an interface between hardware and the user.
 - Main goals: Convenience, Efficiency, Evolution.
 - OS provides services for execution, I/O, file management and security.
-

EVOLUTION OF OPERATING SYSTEM & OS STRUCTURE

PART 1: EVOLUTION OF OPERATING SYSTEM

Operating System ek din me develop nahi hua. Jaise-jaise computer technology advance hui, OS bhi improve hota gaya.

Chalo step-by-step samajhte hain.

1 Early Systems (No Operating System)

Sabse pehle computers me koi OS nahi hota tha.

- User directly machine language me program likhta tha
- Har program manually load hota tha
- Ek time me sirf ek hi program run hota tha

Problem:

- Bahut slow process
- Human effort zyada
- CPU ka time waste hota tha

Is stage par OS ki zarurat feel hui.

2 Batch Operating System

Isme programs ko batch me collect kiya jata tha.

Process:

- User apna program operator ko deta tha
- Operator similar jobs ko group karta tha
- System unhe ek ke baad ek execute karta tha

Features:

- No user interaction
- Automatic job sequencing

Advantages:

- CPU idle time kam hua
- Manual effort kam hua

Disadvantages:

- Error ka result late milta tha
- No debugging during execution

Example: Early IBM Systems

3 Multiprogramming Operating System

Batch system me ek problem thi:

Jab ek program I/O ke liye wait karta tha, CPU idle ho jata tha.

Is problem ko solve karne ke liye multiprogramming aaya.

Concept:

- Multiple programs memory me load hote hain
- Jab ek process wait kare, CPU dusra execute karta hai

Advantages:

- CPU utilization high
- Efficiency better

Disadvantages:

- Complex memory management
 - Scheduling required
-

4 Time-Sharing Operating System

Multiprogramming ko aur improve karke time-sharing system bana.

Concept:

- Multiple users ek system share karte hain
- CPU time small time slices me divide hota hai

Har user ko lagta hai system sirf uske liye kaam kar raha hai.

Advantages:

- Fast response
- Interactive system
- Multi-user support

Example: UNIX

5 Real-Time Operating System (RTOS)

RTOS me response fixed time ke andar dena zaroori hota hai.

Types:

- Hard Real-Time
- Soft Real-Time

Use:

- Traffic control
- Medical systems
- Robotics

Agar delay hua toh system fail ho sakta hai.

6 Distributed Operating System

Isme multiple computers network ke through connected hote hain.

User ko lagta hai sab ek hi system ka part hain.

Advantages:

- Resource sharing
 - Fast processing
 - Reliability
-

PART 2: STRUCTURE OF OPERATING SYSTEM

Operating System ka internal design different tariko se banaya ja sakta hai.

1 Simple Structure (Monolithic Structure)

Isme pura OS ek hi large program hota hai.

Features:

- No clear separation
- All services kernel me hoti hain

Advantages:

- Fast execution
- Simple design

Disadvantages:

- Maintain karna difficult
- Error hone par pura system crash

Example: Early MS-DOS

2 Layered Structure

Isme OS ko layers me divide kiya jata hai.

Har layer ka specific function hota hai.

Structure concept:

Top Layer → User Interface

Middle Layers → System Services

Bottom Layer → Hardware

Rules:

- Har layer sirf niche wali layer se interact karegi

Advantages:

- Easy debugging
- Modular design
- Secure

Disadvantages:

- Performance slow ho sakti hai
-

3 Microkernel Structure

Isme kernel ko minimum rakha jata hai.

Kernel me sirf essential functions:

- CPU scheduling
- Memory management
- IPC

Baaki services user space me run hoti hain.

Advantages:

- More secure
- Flexible
- Crash hone ka risk kam

Disadvantages:

- Communication overhead

Example: Minix, Mach

4 Hybrid Structure

Modern systems me hybrid structure use hota hai.

Isme:

- Monolithic + Microkernel features combine hote hain

Example:

- Windows
 - Linux
-



Important Exam Points

- Batch OS = No interaction
- Multiprogramming = Better CPU utilization
- Time-sharing = Fast response
- RTOS = Fixed response time
- Monolithic = Large single kernel
- Microkernel = Small core + user services
- Layered = Structured design

PROCESS CONCEPT

1 What is a Process?

Process ka matlab hai **program in execution**.

Jab koi program sirf hard disk me stored hota hai, tab wo passive hota hai.

Jab us program ko execute kiya jata hai, tab wo active ho jata hai — aur usko process kehte hain.

Example:

- Agar aap Chrome open karte ho → ek process create hota hai
- Agar aap MS Word bhi open karte ho → dusra process create hota hai

Isliye ek time par system me multiple processes run ho sakte hain.

Important line for exam:

A process is an active entity, whereas a program is a passive entity.

2 Why Process is Important?

Operating System ka main kaam processes ko manage karna hota hai.

Without process management:

- CPU proper use nahi hoga
 - Multiple programs ek saath run nahi ho sakenge
 - System unstable ho jayega
-

3 Components of a Process

Har process ke andar kuch main components hote hain:

1. Text Section (Code Section)

Program ka actual instructions part.

2. Data Section

Global variables aur static variables.

3. Heap

Dynamic memory allocation (malloc, new).

4. Stack

Function calls, local variables, return address store karta hai.

5. Program Counter (PC)

Next instruction ka address store karta hai.

6. CPU Registers

Temporary data aur intermediate results store karte hain.

OS in sab ko track karta hai.

4 Process Control Block (PCB)

Operating System har process ke liye ek special data structure maintain karta hai jise **Process Control Block (PCB)** kehte hain.

PCB me ye information store hoti hai:

1. Process ID (PID)
2. Process State
3. Program Counter
4. CPU Registers
5. Memory Information
6. I/O Status Information
7. Scheduling Information

PCB bahut important hai kyunki context switching ke time OS isi data ka use karta hai.

Exam line:

PCB is a data structure maintained by OS to store process-related information.

5 Process States

Process apni life me different states se pass hota hai.

Main states:

1. New
2. Ready
3. Running
4. Waiting (Blocked)

5. Terminated

6 Process Life Cycle (Detailed Explanation)

Chalo har state ko clearly samjhte hain:

♦ New State

Process create hota hai, lekin abhi ready queue me enter nahi hua.

♦ Ready State

Process execution ke liye ready hai, lekin CPU available nahi hai.

♦ Running State

Process currently CPU use kar raha hai.

♦ Waiting State

Process kisi event ka wait kar raha hai (jaise I/O complete hone ka).

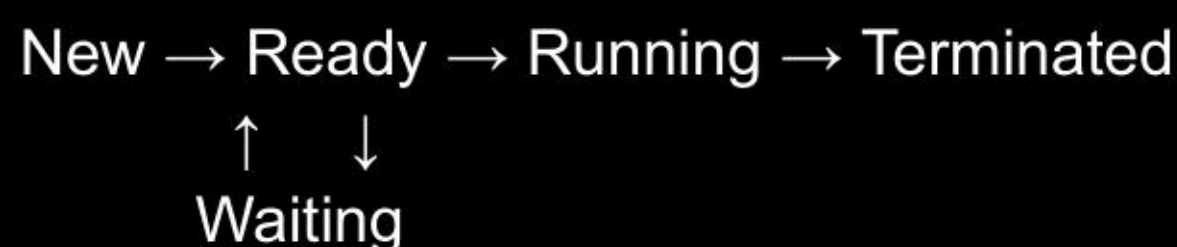
♦ Terminated State

Process execution complete ho gaya hai.

State Transition Explanation

1. New → Ready
Jab process admission milta hai
2. Ready → Running
CPU assign hota hai
3. Running → Waiting
I/O request karta hai
4. Waiting → Ready
I/O complete
5. Running → Terminated
Execution finish

Exam me diagram aise likh sakte ho:



7 Context Switching

Jab CPU ek process se dusre process par switch karta hai, use context switching kehte hain.

Steps:

1. Current process ka state PCB me save hota hai
2. Next process ka state load hota hai

Context switching fast hona chahiye warna performance slow ho sakti hai.

8 Process vs Program (Important Difference)

Program	Process
Passive entity	Active entity
Hard disk me stored	Memory me loaded
Execution nahi karta	Execution karta hai

9 Process vs Thread (Introduction Level)

Process:

- Independent execution unit
- Separate memory space

Thread:

- Process ka part
- Shared memory space

Detailed thread concept next response me cover karenge.

THREADS, MULTITHREADING, IPC & CRITICAL SECTION

PART 1: THREAD CONCEPT

1 What is a Thread?

Thread ek **lightweight execution unit** hai jo process ke andar run karta hai.

Simple words me:

Process ek ghar hai.

Threads us ghar ke rooms ki tarah hain jahan alag-alag kaam ho rahe hote hain.

Ek process ke andar multiple threads ho sakte hain.

2 Why Threads are Needed?

Agar har chhote kaam ke liye naya process banaya jaye, toh:

- Memory zyada lagegi
- Context switching costly hoga
- System slow hoga

Isliye threads use kiye jate hain.

3 Components of a Thread

Har thread ke paas hota hai:

- Thread ID
- Program Counter
- Registers
- Stack

Lekin threads same process ke:

- Code section
- Data section

- Heap

share karte hain.

Isi wajah se threads fast hote hain.

4 Advantages of Multithreading

1. Responsiveness
Application freeze nahi hota
 2. Resource Sharing
Same memory share karte hain
 3. Economy
Thread creation cheap hoti hai
 4. Utilization of Multi-core CPU
Parallel execution possible
-

PART 2: MULTITHREADING MODELS

OS threads ko manage karne ke liye different models use karta hai.

1 Many-to-One Model

Multiple user threads → single kernel thread.

Problem:

Agar ek thread block ho gaya, sab block ho jayenge.

Parallelism possible nahi hota.

2 One-to-One Model

Har user thread ka ek kernel thread hota hai.

Advantage:

True parallelism possible.

Disadvantage:

Zyada threads → zyada overhead.

3 Many-to-Many Model

Multiple user threads → multiple kernel threads.

Advantage:

Flexible aur efficient.

Ye model modern systems me use hota hai.

PART 3: INTER-PROCESS COMMUNICATION (IPC)

1 What is IPC?

IPC ka matlab hai **processes ke beech communication**.

Kyuki har process ka apna memory space hota hai, directly data share nahi kar sakte.

Isliye OS IPC mechanisms provide karta hai.

2 IPC ke Two Main Methods

♦ 1. Shared Memory

- OS ek shared memory region create karta hai
- Multiple processes usko access karte hain

Fast method hai.

Problem:

Synchronization required.

♦ 2. Message Passing

- Processes messages send aur receive karte hain

- OS ke through communication hota hai

Slow compared to shared memory, but safe.

3 IPC Mechanisms Examples

- Pipes
 - Message Queues
 - Shared Memory
 - Sockets
-

PART 4: CRITICAL SECTION PROBLEM

1 What is Critical Section?

Critical Section program ka wo part hai jahan shared resource access hota hai.

Example:

Agar do processes same bank account balance update karein, toh error ho sakta hai.

2 Problem

Agar ek se zyada process ek hi time par critical section me enter karein, toh:

- Data inconsistency
- Wrong output

Isliye synchronization required hai.

3 Requirements of Critical Section Solution

Solution ko 3 conditions satisfy karni chahiye:

1. Mutual Exclusion

Ek time me sirf ek process critical section me ho.

2. Progress

Agar koi process inside nahi hai, toh decision delay nahi hona chahiye.

3. Bounded Waiting

Koi process infinite wait nahi kare.

4 Synchronization Tools

OS kuch tools provide karta hai:

♦ Mutex

Lock mechanism. Ek process lock karega, baaki wait karenge.

♦ Semaphore

Integer variable use karta hai synchronization ke liye.

Types:

- Binary Semaphore
- Counting Semaphore

♦ Monitor

High-level synchronization mechanism.

Important Exam Points

- Thread = Lightweight process
 - Threads share memory
 - Many-to-One → No parallelism
 - One-to-One → Parallelism possible
 - IPC = Shared Memory + Message Passing
 - Critical Section needs Mutual Exclusion
-

CPU SCHEDULING & DEADLOCK (DETAILED THEORY)

PART 1: CPU SCHEDULING

1 What is CPU Scheduling?

CPU Scheduling ka matlab hai:

Jab multiple processes ready state me hote hain, tab CPU kisko pehle execute karega — ye decide karna.

Operating System ka scheduler ye decision leta hai.

Simple example:

Agar 5 processes ready queue me hain, toh CPU ek time me sirf ek ko execute karega.

Baaki wait karenge.

Isliye scheduling algorithm zaroori hai.

2 Objectives of CPU Scheduling

Scheduling ka goal hota hai:

- CPU utilization maximum ho
 - Waiting time minimum ho
 - Turnaround time kam ho
 - Response time fast ho
 - Fairness maintain ho
-

3 Important Scheduling Terms

♦ CPU Burst

Time period jab process CPU use karta hai.

♦ Waiting Time

Total time jo process ready queue me wait karta hai.

- ◆ **Turnaround Time**

Completion time – Arrival time

- ◆ **Response Time**

First response milne tak ka time.

4 CPU Scheduling Algorithms

1. FCFS (First Come First Serve)

Jo process pehle aata hai, wo pehle execute hota hai.

Type:

Non-preemptive

Advantage:

- Simple
- Easy to implement

Disadvantage:

- Convoy effect (short process ko long process ka wait karna padta hai)
 - Waiting time high ho sakta hai
-

2. SJF (Shortest Job First)

Sabse chhota CPU burst wala process pehle execute hoga.

Type:

Non-preemptive

(Preemptive version: Shortest Remaining Time First)

Advantage:

- Minimum average waiting time

Disadvantage:

- Next CPU burst predict karna difficult
- Starvation ho sakta hai

3. Priority Scheduling

Har process ko priority di jati hai.

High priority process pehle execute hota hai.

Type:

Preemptive ya Non-preemptive dono ho sakta hai.

Advantage:

- Important processes ko preference

Disadvantage:

- Low priority processes starvation face kar sakte hain

Solution:

Aging technique use karte hain.

4. Round Robin (RR)

Time-sharing system me use hota hai.

Har process ko fixed time slice (quantum) milta hai.

Agar process quantum me finish nahi hua, toh next process ko chance milta hai.

Advantage:

- Fair scheduling
- Response time better

Disadvantage:

- Quantum size correct hona chahiye
 - Zyada context switching overhead
-

PART 2: DEADLOCK

1 What is Deadlock?

Deadlock ek situation hai jahan do ya zyada processes ek dusre ka wait kar rahe hote hain, aur koi bhi aage nahi badh pata.

Simple example:

Process P1 → Resource R1 hold kar raha hai, R2 ka wait kar raha hai

Process P2 → R2 hold kar raha hai, R1 ka wait kar raha hai

Dono wait karte rahenge. System freeze ho sakta hai.

2 Necessary Conditions for Deadlock

Deadlock tabhi hoga jab ye 4 conditions ek saath present ho:

1. Mutual Exclusion

Resource ek time me sirf ek process use kare.

2. Hold and Wait

Process ek resource hold kare aur dusra wait kare.

3. No Preemption

Resource zabardasti wapas nahi liya ja sakta.

4. Circular Wait

Processes circular chain me ek dusre ka wait karein.

Agar inme se ek condition bhi remove kar do, deadlock nahi hoga.

3 Deadlock Prevention

Deadlock hone se pehle hi rok dena.

Methods:

- Mutual exclusion avoid karna (possible nahi usually)
- Hold and wait avoid karna
- Preemption allow karna
- Circular wait break karna

Example:

Resource numbering technique use karte hain.

4 Deadlock Avoidance

System resource allocation aise karta hai ki unsafe state na aaye.

Example:

Banker's Algorithm

Concept:

Safe state maintain karna.

5 Deadlock Detection and Recovery

Kabhi-kabhi deadlock allow karte hain, baad me detect karte hain.

Detection:

- Resource Allocation Graph
- Detection algorithm

Recovery methods:

1. Process terminate karna
 2. Resource preemption
-

Important Exam Points

- FCFS = Simple but high waiting time
 - SJF = Minimum waiting time
 - Round Robin = Time-sharing system
 - Deadlock = Circular waiting situation
 - 4 Necessary conditions must occur together
 - Prevention, Avoidance, Detection are main approaches
-

MEMORY MANAGEMENT – PART 1

1 Introduction to Memory Management

Memory Management ka matlab hai:

Operating System RAM ko manage karta hai aur decide karta hai:

- Kaunsa process kitni memory use karega
- Memory kab allocate hogi
- Memory kab free hogi

Agar memory properly manage nahi hui, toh system crash ho sakta hai.

2 Contiguous Memory Allocation

Contiguous allocation me process ko memory ek continuous block me allocate ki jati hai.

Example:

Agar process ko 100 KB chahiye, toh 100 KB ka continuous space milega.

Types of Contiguous Allocation

1. Fixed Partitioning

Memory ko fixed size partitions me divide kiya jata hai.

Problem:

- Agar process chhota hai toh memory waste hogi
 - Large process fit nahi hoga
-

2. Variable Partitioning

Memory dynamic tarike se allocate hoti hai.

Process ke size ke hisab se partition banta hai.

Problem:

- External fragmentation ho sakta hai
-

3 Fragmentation

Fragmentation ka matlab hai memory waste hona.

Do types hote hain:

◆ Internal Fragmentation

Jab allocated memory ka kuch part unused reh jata hai.

Example:

Agar partition size 100 KB hai aur process ko 70 KB chahiye, toh 30 KB waste ho gaya.

Ye waste memory allocated block ke andar hoti hai.

◆ External Fragmentation

Jab free memory chhote-chhote blocks me scattered hoti hai.

Total free memory enough hai, lekin continuous nahi hai.

Isliye large process allocate nahi ho pata.

Solution for External Fragmentation

Compaction:

Memory ko rearrange karke free space ek jagah collect karna.

4 Paging

Paging ek memory management technique hai jo fragmentation problem solve karta hai.

Concept:

- Physical memory ko fixed size blocks me divide kiya jata hai → Frames
- Logical memory ko fixed size blocks me divide kiya jata hai → Pages

Process ke pages ko kisi bhi free frame me load kiya ja sakta hai.

Isliye contiguous memory ki zarurat nahi hoti.

Paging ke Advantages

- External fragmentation eliminate hoti hai
 - Efficient memory utilization
-

Disadvantage

- Internal fragmentation ho sakta hai
 - Page table maintain karna padta hai
-

5 Segmentation

Segmentation logical memory division par based hai.

Program ko logical segments me divide kiya jata hai:

- Code segment
- Data segment
- Stack segment

Har segment ka size different hota hai.

Address format:

Segment number + Offset

Advantages of Segmentation

- Logical view provide karta hai
- Better protection
- Easy sharing

Disadvantage

- External fragmentation ho sakta hai
-

6 Segmentation with Paging

Is technique me segmentation aur paging dono combine hote hain.

Steps:

1. Program ko segments me divide karte hain
2. Har segment ko pages me divide karte hain

Isse:

- Logical structure maintain hota hai
- External fragmentation reduces hoti hai

Modern systems me ye approach use hoti hai.



Important Exam Points

- Contiguous allocation = Continuous block
 - Internal fragmentation = Waste inside the allocated block
 - External fragmentation = Scattered free space
 - Paging removes external fragmentation
 - Segmentation supports logical division
 - Segmentation + Paging = Combined advantage
-

VIRTUAL MEMORY, DEMAND PAGING & PAGE REPLACEMENT (DETAILED)

1 Virtual Memory

Virtual Memory ek technique hai jisme process ko lagta hai ki uske paas bahut badi memory available hai, chahe actual RAM chhoti ho.

Concept:

- Program ka pura part RAM me load nahi hota
- Sirf required pages load hote hain
- Baaki pages secondary storage (hard disk) me rehte hain

Is technique se:

- Large programs run ho sakte hain
 - Multiprogramming improve hota hai
 - RAM ka efficient use hota hai
-

2 Demand Paging

Demand Paging virtual memory ka part hai.

Isme page tabhi memory me load hota hai jab uski actual demand hoti hai.

Steps:

1. CPU page access karta hai
 2. Agar page memory me nahi hai → Page Fault
 3. OS required page disk se load karta hai
 4. Execution continue hota hai
-

Page Fault

Page Fault error nahi hai.

Ye ek signal hai ki required page RAM me available nahi hai.

Advantages of Demand Paging

- Memory saving
 - Fast program start
 - Less I/O required
-

Disadvantages

- Page fault handling me delay
 - Excess page faults → Thrashing
-

3 Page Replacement Algorithms

Jab memory full ho jati hai aur naya page load karna hota hai, toh kisi existing page ko remove karna padta hai.

Is process ko Page Replacement kehte hain.

1 FIFO (First In First Out)

Sabse pehle jo page memory me aaya tha, use remove kar diya jata hai.

Working:

Queue follow karta hai.

Advantage:

- Simple implementation

Disadvantage:

- Belady's anomaly ho sakti hai
(Frames badhane par bhi page faults badh sakte hain)
-

2 LRU (Least Recently Used)

Jo page sabse zyada time se use nahi hua, use remove kiya jata hai.

Concept:

Recent usage history maintain karte hain.

Advantage:

- Better performance than FIFO

Disadvantage:

- Implementation costly
 - Hardware support required
-

3 Optimal Page Replacement

Jo page future me sabse late use hoga, use remove karte hain.

Advantage:

- Minimum page faults
- Best theoretical algorithm

Disadvantage:

- Future predict nahi kar sakte
 - Practical implementation possible nahi
-

4 Difference: Demand Paging vs Page Replacement

Demand Paging	Page Replacement
Page ko load karna	Page ko remove karna
Page fault par hota hai	Memory full hone par hota hai
Virtual memory ka part	Memory management strategy

Simple words me:

Demand paging = Page kab load karna hai

Page replacement = Kaunsa page remove karna hai

5 Thrashing

Jab system continuously page faults handle karta rahe aur actual execution kam ho jaye, use Thrashing kehte hain.

Reason:

- Zyada multiprogramming
- Insufficient frames

Solution:

- Degree of multiprogramming kam karo
 - Working set model use karo
-



Important Exam Points

- Virtual Memory allows execution of large programs
 - Demand Paging loads page only when needed
 - Page Fault occurs when page not in memory
 - FIFO simple but may cause Belady's anomaly
 - LRU better but costly
 - Optimal gives minimum page faults
 - Thrashing reduces system performance
-

FILE SYSTEM, DISK MANAGEMENT, SECURITY & ADVANCED OS CONCEPTS

PART 1: FILE SYSTEM

1 What is a File?

File ek collection of related information hai jo secondary storage (hard disk) me store hoti hai.

Operating System files ko:

- Create karta hai
 - Delete karta hai
 - Read/Write karta hai
 - Organize karta hai
-

2 File Allocation Methods

File ko disk me store karne ke liye different methods use hote hain.

◆ 1. Contiguous Allocation

File ke saare blocks continuous memory location me store hote hain.

Example:

Agar file 5 blocks ki hai, toh 5 continuous blocks milenge.

Advantages:

- Fast access
- Simple implementation

Disadvantages:

- External fragmentation
 - File size increase karna difficult
-

◆ 2. Linked Allocation

File ke blocks disk me kahin bhi ho sakte hain.
Har block next block ka pointer store karta hai.

Advantages:

- No external fragmentation
- File easily grow kar sakti hai

Disadvantages:

- Random access slow
 - Pointer overhead
-

◆ 3. Indexed Allocation

Ek separate index block maintain hota hai jo file ke saare block addresses store karta hai.

Advantages:

- Direct access possible
- No external fragmentation

Disadvantages:

- Index block space consume karta hai
-

PART 2: DIRECTORY IMPLEMENTATION

Directory files ko organize karne ke liye use hoti hai.

Types:

1 Single-Level Directory

Sab files ek hi directory me.

Problem:

- Name conflict
-

2 Two-Level Directory

Har user ki alag directory.

Better organization.

3 Tree-Structured Directory

Root directory se multiple subdirectories.

Modern OS me ye use hota hai.

PART 3: FREE SPACE MANAGEMENT

Disk me free blocks ko track karna zaroori hai.

Methods:

1. Bit Map
Har block ke liye ek bit maintain hoti hai.
 2. Linked List
Free blocks linked list me store hote hain.
 3. Grouping
Multiple free blocks ek saath store hote hain.
-

PART 4: DISK SCHEDULING ALGORITHMS

Disk scheduling ka goal hai:

- Seek time kam karna
 - Response time improve karna
-

1 FCFS

Jo request pehle aayi, pehle serve hogi.

Simple but inefficient.

2 SSTF (Shortest Seek Time First)

Head ke nearest request ko pehle serve karte hain.

Fast but starvation possible.

3 SCAN (Elevator Algorithm)

Disk head ek direction me move karta hai, phir reverse hota hai.

Better performance.

4 C-SCAN

Head sirf ek direction me serve karta hai, phir start par return karta hai.

Uniform waiting time.

PART 5: OS SECURITY

Operating System security ensure karta hai:

- User authentication
- Access control
- Data protection
- Virus protection

Security mechanisms:

- Password
 - Access Control List (ACL)
 - Encryption
-

PART 6: VIRTUAL MACHINES

Virtual Machine ek software environment hai jo real computer jaisa behave karta hai.

Hypervisor use hota hai VM create karne ke liye.

Advantages:

- Resource sharing
- Isolation
- Testing safe environment me

Example:

VMware, VirtualBox

PART 7: CLOUD OPERATING SYSTEM

Cloud OS distributed computing environment me use hota hai.

Features:

- Remote access
- Resource pooling
- Scalability
- On-demand services

Cloud platforms example:

- AWS
 - Microsoft Azure
-



Final Revision Points

- Contiguous, Linked, Indexed = File allocation methods
 - Tree structure directory modern systems me use hoti hai
 - Disk scheduling improves seek time
 - VM provides isolated environment
 - Cloud OS supports distributed systems
-